

Le patron de conception Fabrique abstraite

Durée : environ 1 h

Crédit : <https://www.math.univ-paris13.fr/~chaussar/>

1 Instructions générales

1. Téléchargez l'archive zip associée à cet énoncé sur la plateforme pédagogique.
2. Ouvrez le projet dans l'IDE IntelliJ IDEA (ou autre).
3. Si vous ne l'avez pas encore fait, configurez votre IDE pour n'accepter que des attributs avec un identifiant commençant pas un '_'. (*Dans IntelliJ IDEA, menu Settings::Editor::Code Style::Java, ajoutez le caractère '_' dans field et static field. Puis, dans le menu Settings::Editor::inspection::Java::Naming Conventions ajoutez la vérification de Field naming convention (instance field et static field) avec l'expression régulière : `_[a-z][A-Za-z\d]*`.*)
4. Exécutez le programme pour vérifier que tout fonctionne correctement.

2 Données de l'exercice

Le code fourni dans l'archive permet d'afficher le nom d'un fichier à partir d'un chemin. Le code définit une classe `Main` avec une méthode statique :

```
public class Main {  
    public static void parseFilename(String path) {  
        int index = path.lastIndexOf("\\");  
        String r = path.substring(index + 1);  
        System.out.println(r);  
    }  
}
```

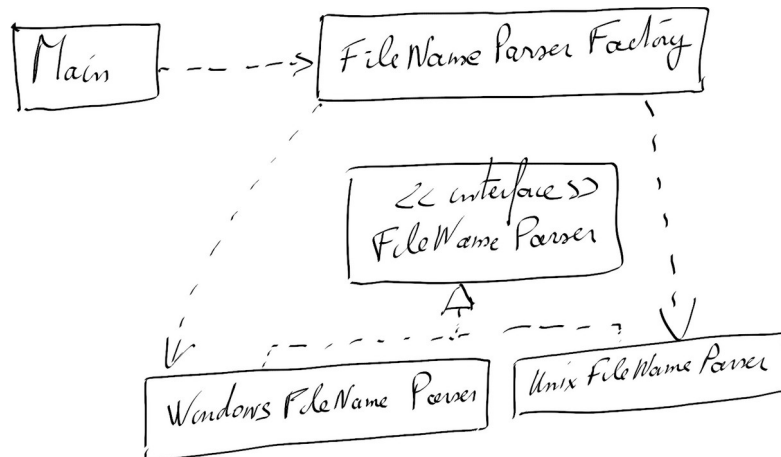
Si vous exécutez ce programme en lui passant en paramètre "C:\\Windows\\hello.dll", il doit vous renvoyer "hello.dll".

3 Exercice 1 : Fabrique simple

On souhaite étendre ce programme aux chemins Unix (eg, "/user/share/hello.rc").

Pour ce faire, vous devez créer une interface `FileNameParser` et deux sous-classes `WindowsFileNameParser` et `UnixFileNameParser` qui l'implémentent. Ensuite, vous devez créer une fabrique `FileNameParserFactory` qui va créer des

FileNameParser (Windows ou Unix) dans la fonction `main()` à partir d'une méthode statique `createParse(String os)` dont le paramètre `os` vaut "unix" ou "windows".



Faites les tests sur les noms de dossier "C:\\Windows\\hello.dll" sous Windows et "/unix/hello.so" sous Unix.

4 Exercice 2 : Fabrique multiple

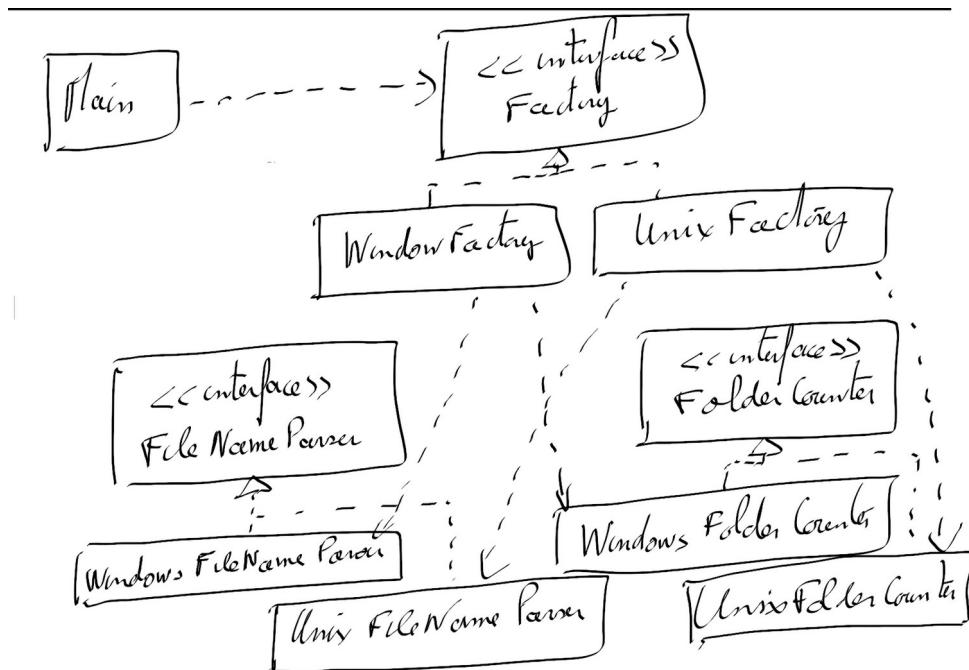
On veut maintenant compter le nombre de dossiers qu'il faut parcourir depuis la racine pour trouver le fichier. Ajoutez une interface `FolderCounter` avec une méthode `count()`, et deux sous classes qui l'implémenteront (une Windows, et une Unix). Pour compter les dossiers, utilisez la fonction `split()` des Strings avec l'expression régulière "\\\\" ou "/" selon l'OS.

Ensuite, il va falloir créer une nouvelle fabrique `FileNameFactory` qui possède une méthode `createParsers(String type)` qui crée les deux types d'objets (un `FileNameParser` et un `FolderCounter`) en fonction du type d'OS. Les deux objets sont accessibles dans deux variables de classe publiques de `Factory` (`fileParser` et `folderCount`).

5 Exercice 3 : Une fabrique abstraite

Le problème de cette implémentation est que, si on veut encore ajouter une fonctionnalité, il faut changer le code de la fabrique ce qui est contraire au principe d'ouverture fermeture.

Vous devez donc faire de la fabrique une interface et créer deux sous-fabriques qui l'implémentent (une pour Unix et une pour Windows). Le diagramme de classe devrait ressembler à ceci :



Pour éviter que, dans la classe `Main`, on ait à choisir explicitement quelle fabrique implémenter, et pour éviter que le `Main` ait connaissance de toutes nos différentes fabriques, vous devez faire une fabrique de fabrique qui à partir d'une chaîne "Windows" ou "Unix" construit la bonne fabrique. Vous pouvez, ensuite, utiliser la fabrique créée pour créer un `Parser` ou un `Counter` sur l'exemple du chemin.

6 Exercice 4 : Vérification du principe d'ouverture sur les implémentations

On veut ajouter le support des chemins Macintosh. Le code des deux fonctions `parseFilename()` et `countFolder()` est le même que pour Unix, mais, pour l'exercice, on dupliquera le code. Faites les modifications nécessaires et vérifiez que cela se fait sans modifier le code métier.

7 Exercice 5 : Vérification du principe d'ouverture sur les fonctionnalités

On souhaite ajouter une nouvelle fonctionnalité `parseFolder()` qui donne le nom du répertoire source qui contient notre fichier (par exemple "C:\\Windows" pour notre exemple de départ). Faites les modifications nécessaires et vérifiez que cela se fait sans modifier le code métier.